

PPSDN 2026

SDN Intro

Institut für Telematik, Prof. Zitterbart



© Peter Baumung

5

Software
Defined
Networks

5.1

Basics and Architecture

5.2

SDN Workflow in Practice

5.3

(OpenFlow)

5.4

The Power of Abstraction

5.5

SDN Challenges

5.6

SDN Use Cases

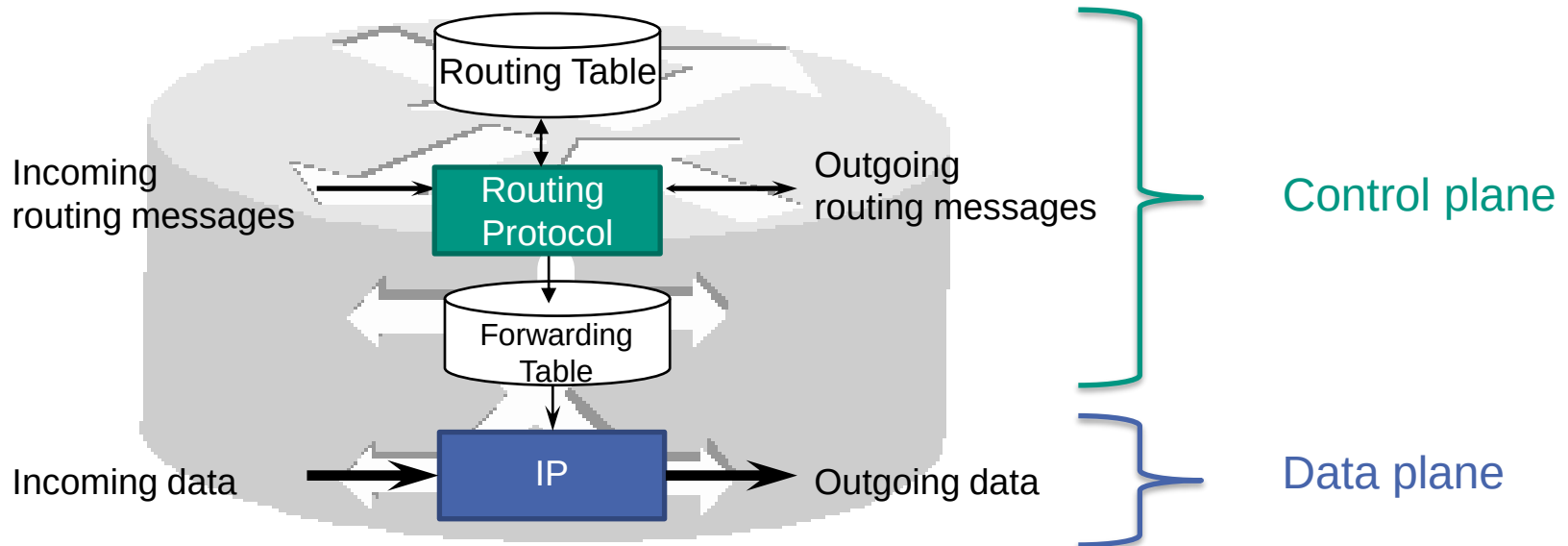
5.7

Tools

WHAT IS SDN?

High Level View on Traditional IP Networks

■ Recap: abstract view on an IP router



■ Control plane

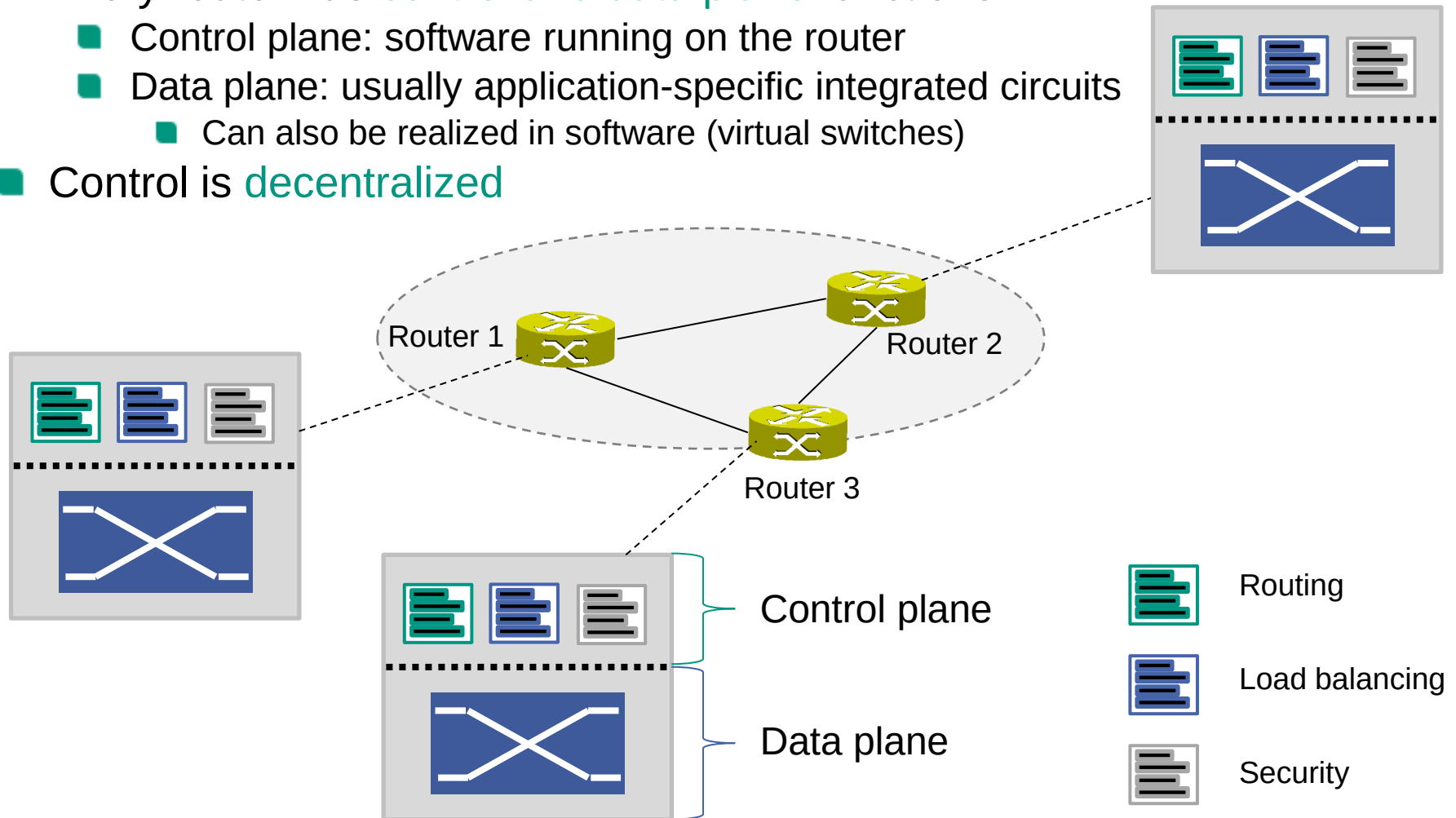
- Exchange of routing messages for calculation of routes ...
- Additional tasks, such as load balancing, access control, ...

■ Data plane

- Forwarding of packets at layer 3

High Level View on Traditional IP Networks

- Every router has **control and data plane** functions
 - Control plane: software running on the router
 - Data plane: usually application-specific integrated circuits
 - Can also be realized in software (virtual switches)
- Control is **decentralized**



Observations

- So why did we build IP networks like this?
 - Proven concept for 30+ years
 - High reliability → self-organisation, high redundancy
 - Highly optimized hardware → very fast

- Limitations
 - Manufacturer-specific management interfaces
 - Difficult (and often impossible) to introduce new functions
 - Complex, highly qualified operators required
 - Expensive (at least for core routers)

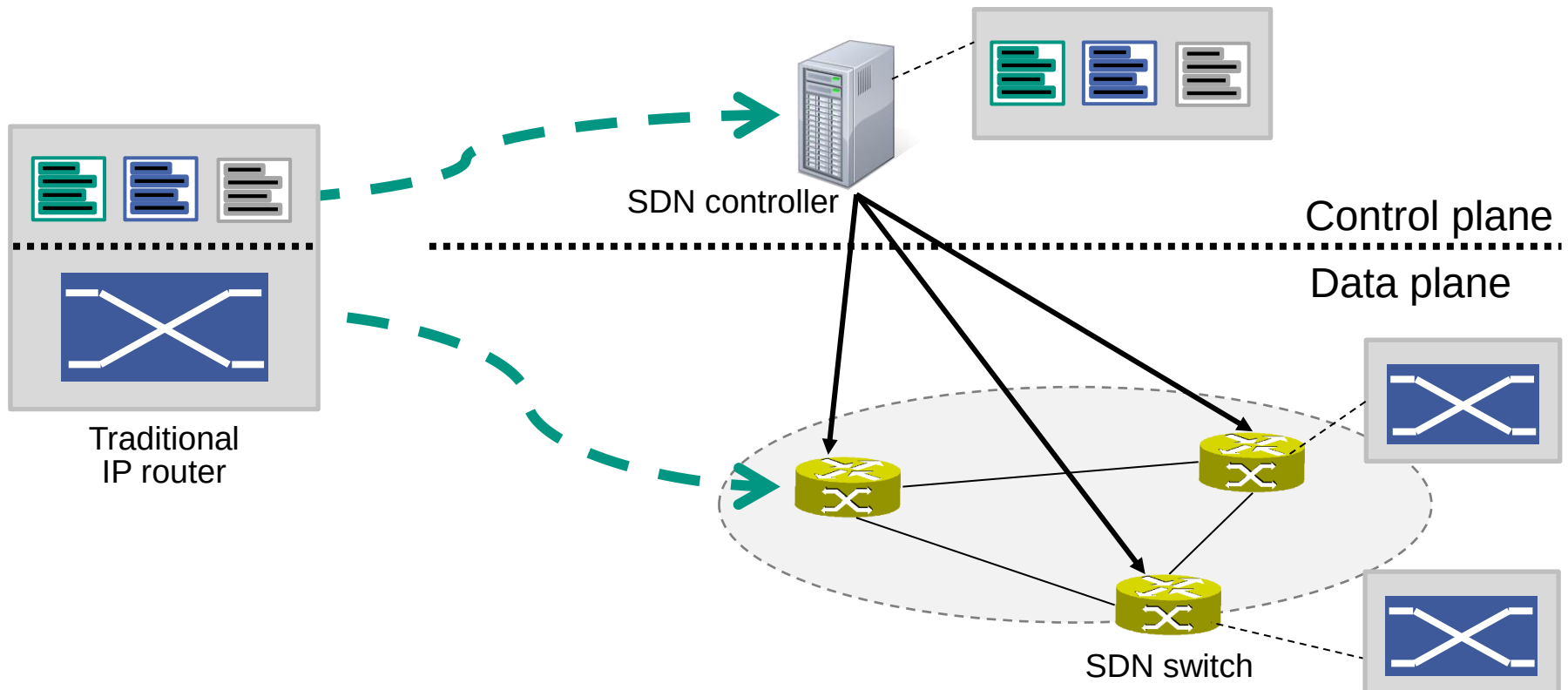
- Current trend: software-defined networks
 - Increase flexibility
 - Decrease dependencies on hardware and manufactures
 - Commercial off-the-shelf switches (cheaper)



Limited
flexibility for
network
operators

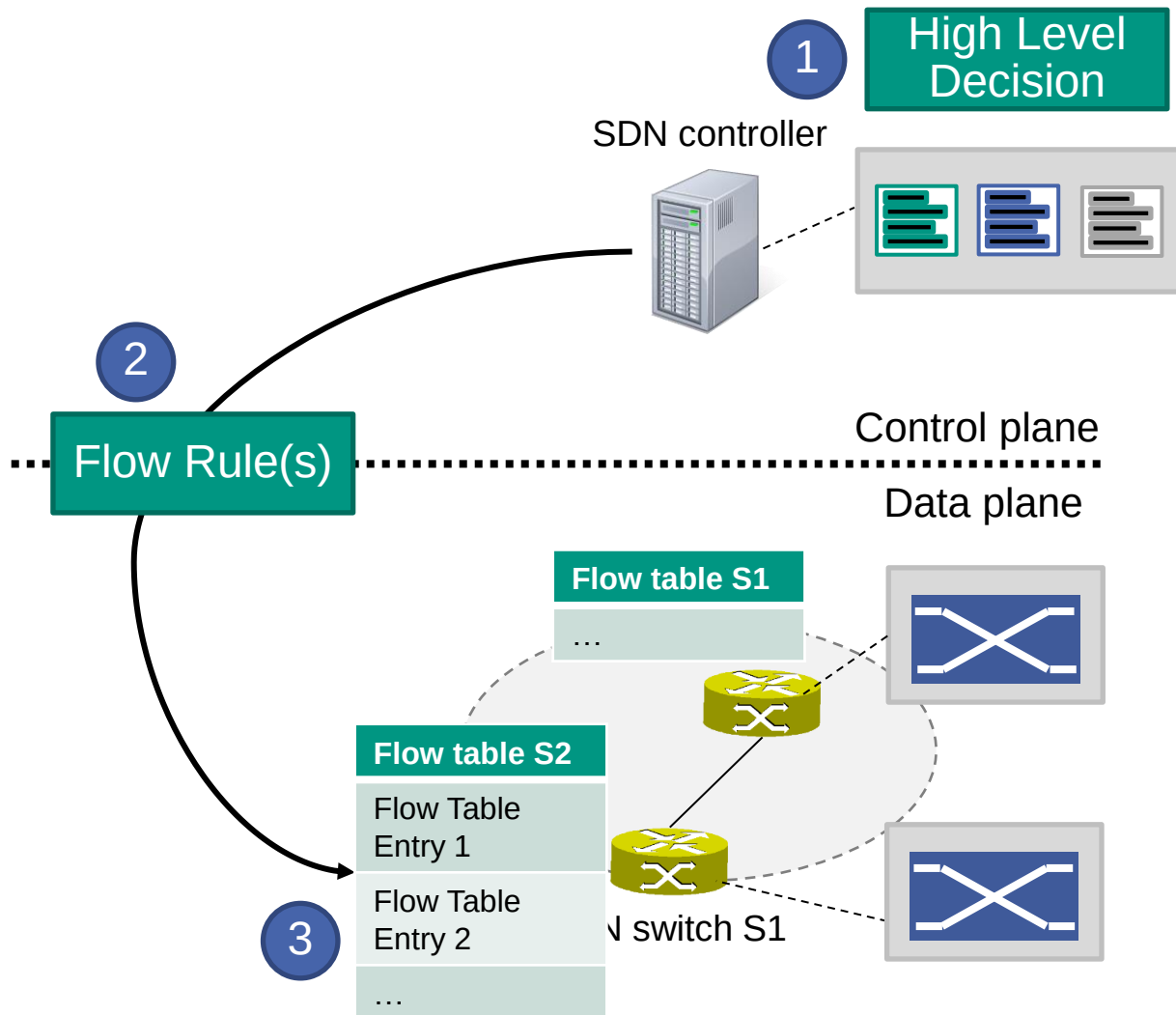
Characteristics of Software-Defined Networks

- **Separation** of control plane and data plane
 - Control functionality resides on a logically centralized **SDN controller**
 - Controller is executed on commodity hardware
 - Reduces need for specialized routing hardware
 - Data plane consists of **simple packet processors (SDN switches)**



HOW DOES IT WORK?

Flow Rule and Flow Table Entries



Two Modes of Operation

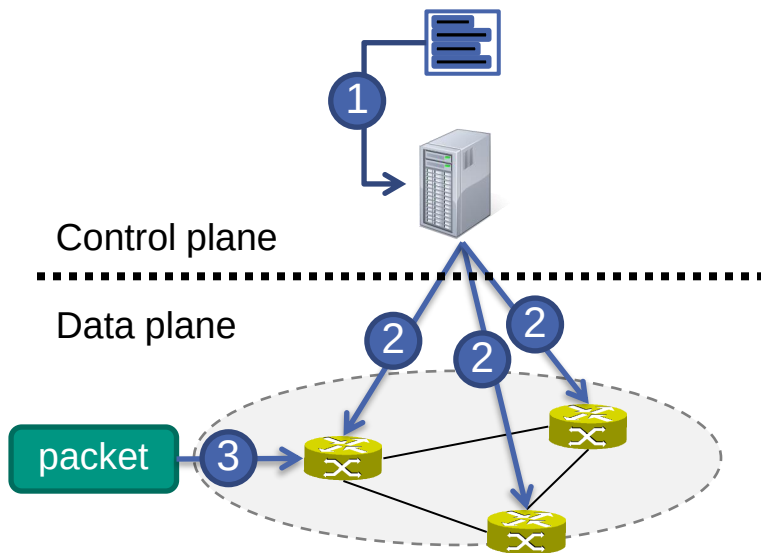
■ Proactive flow programming

- Flow rules are programmed before first packet of flow arrives

■ Reactive flow programming

- Flow rules are programmed in reaction to the receipt of the first packet of a flow

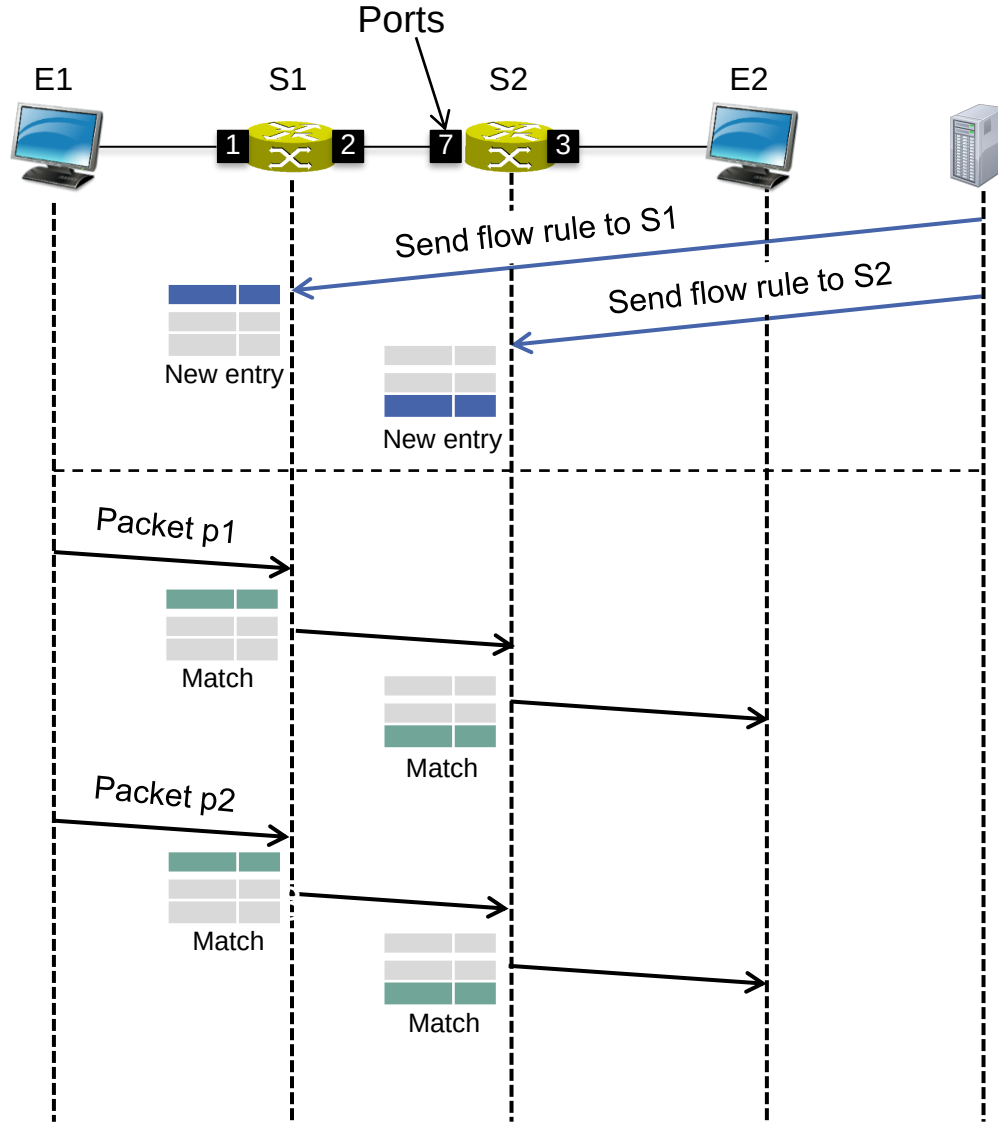
Proactive Flow Programming



“Flow rules are programmed
before first packet of flow
arrives”

1. A control application (App) makes a decision for the packet beforehand (for all switches)
2. Flow rules are sent to the switches and new flow table entries is added to the flow tables
3. Lookup in flow table → Match → Packet is NOT forwarded to controller

Example: Proactive Flow Programming



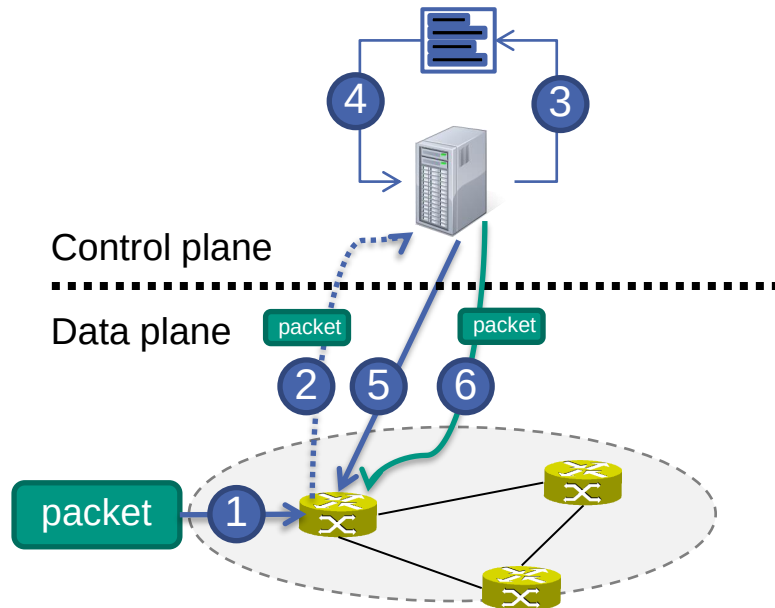
Controller programs flow table entries proactively into switch S1 and S2 before packet p1 and p2 arrive

Switch S1 and S2 forward packets p1 and p2 based on programmed flow table entries

Flow table of S1

Match Fields	Action
IPv4Dst = 4.2.4.15	Output 2
...	...
*	Controller

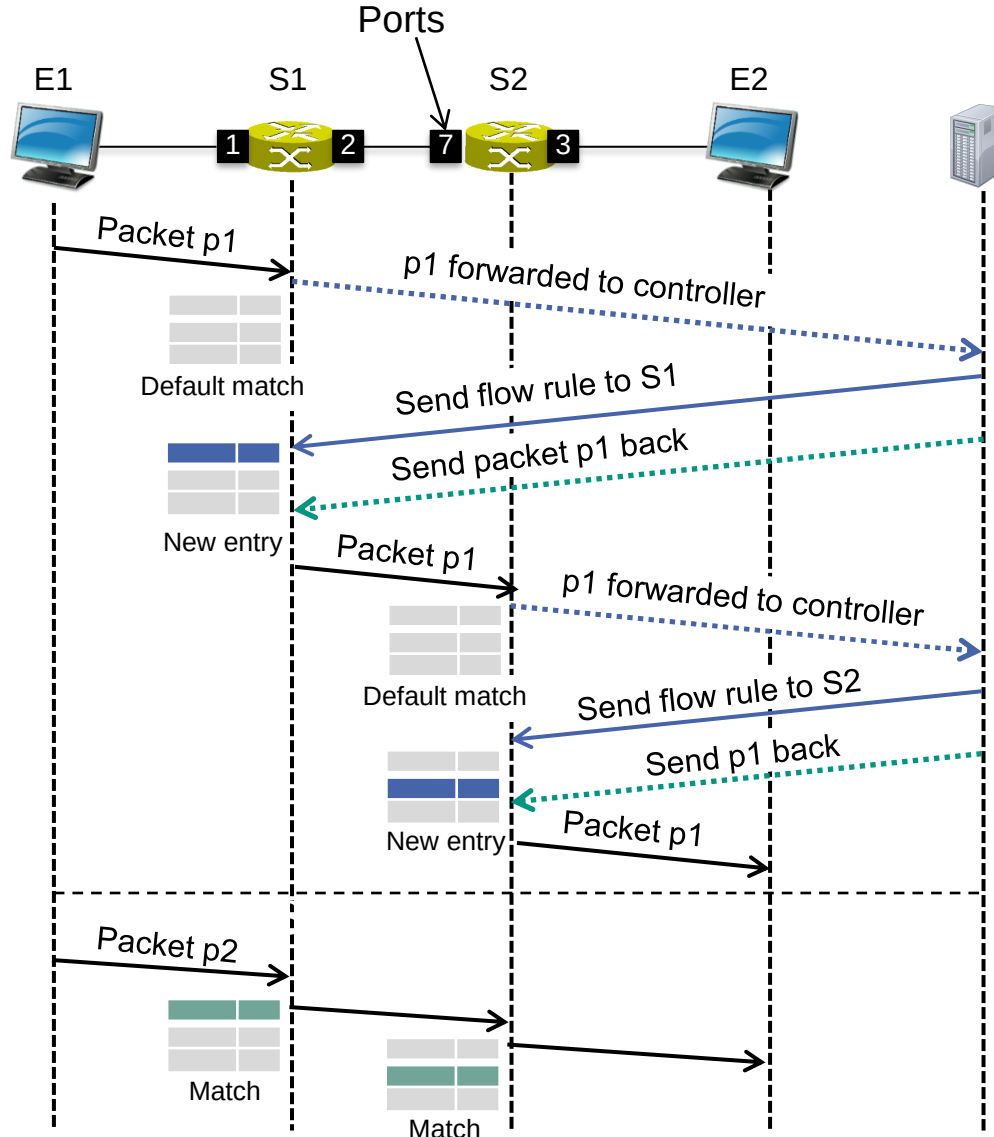
Reactive Flow Programming



„Flow rules are programmed **in reaction** to the receipt of the first packet of a flow“

1. Packet arrives at the switch
2. Lookup in flow table → Mismatch → Packet forwarded to controller
3. Controller notifies a control application (App) responsible for deciding what to do with the packet
4. App has come to a decision. Here: a new flow rule has to be installed
5. Flow rule is sent to the switch and a new flow table entry is added to the flow table
6. Packet is re-injected into the switch

Example: Reactive Flow Programming



Controller programs flow table entries reactively into switch S1 and S2

Flow table of S1

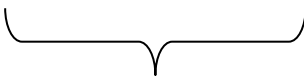
Match Fields	Action
IPv4Dst = 4.2.4.15	Output 2
...	...
*	Controller

Switch S1 and S2 forward the packet based on programmed flow table entries

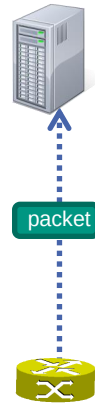
Three Important Interactions



Flow rule sent
to a switch
which results
in a new flow
table entry



Used in the
reactive and
proactive case



Packet forwarded
to controller,
e.g., after
mismatch in flow
table



Packet re-injected
to a switch,
e.g., to not loose a
packet that was
sent to the
controller



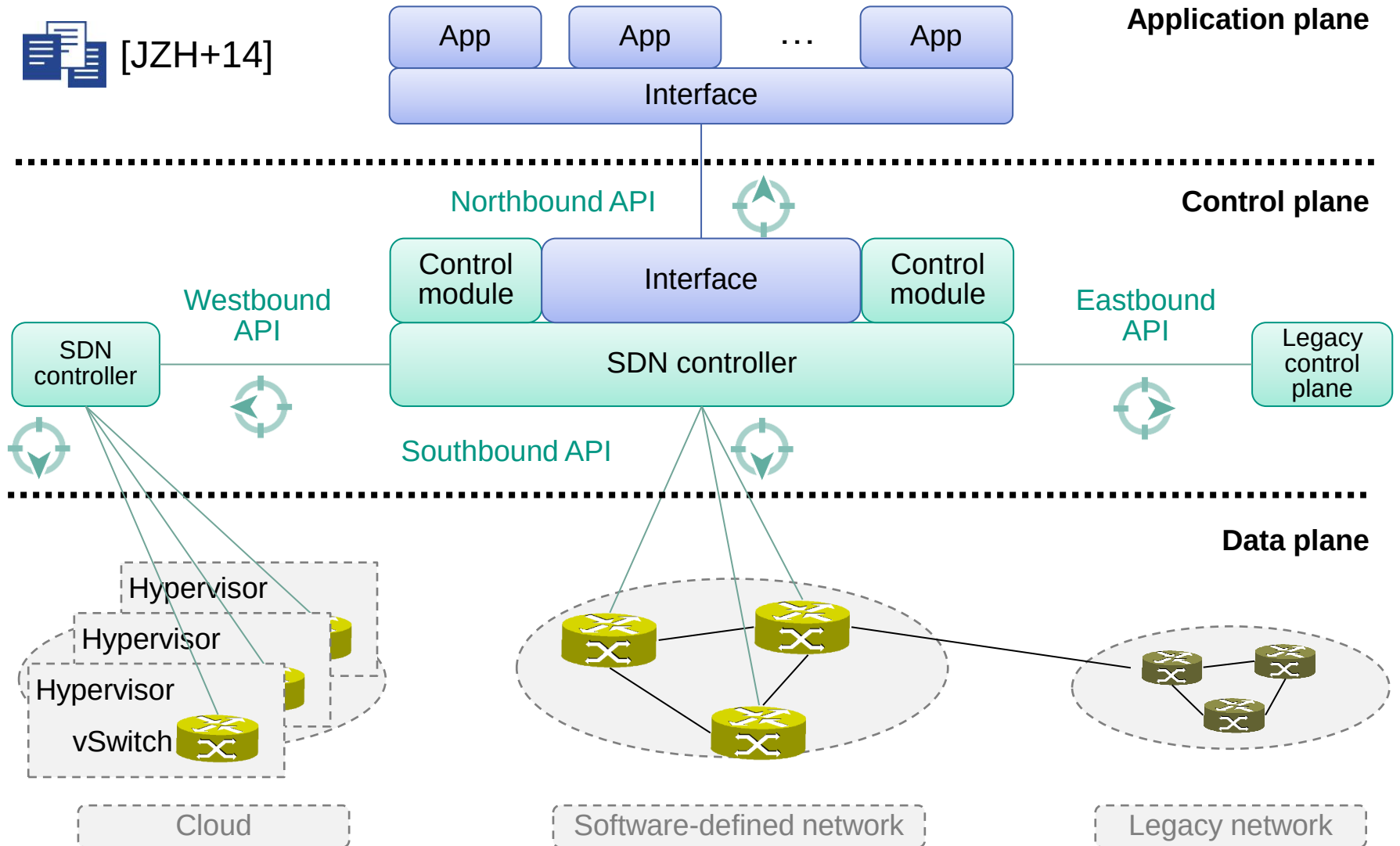
Primarily used in the
reactive case

SDN ARCHITECTURE

SDN Architecture



[JZH+14]



■ Application Plane

- Network apps perform network control and management tasks
- Interacts via **northbound API** with control plane

■ Control Plane

- Control tasks are „**outsourced**“ from the data plane to the logically centralized control plane
 - E.g., standard tasks such as topology detection, ARP ...
- More complex tasks can be delegated to the application plane
 - E.g., routing decisions, load balancing ...

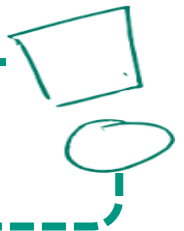
■ Data Plane

- Responsible for **packet forwarding / processing**
- SDN switches are relatively simple devices
 - Efficient implementations in hardware (ASIC) or in software (virtual switches)
 - Supports basic operations such as match, forward, drop
- Interacts via **southbound API** with control plane

SDN Architecture: Interfaces

- **Northbound API:** between controller and network apps 
 - Exposes **control plane functions** to apps
 - Abstract from details, apps can operate on **consistent network view**
- **Southbound API:** between controller and switches 
 - Exposes **data plane functions** to controller
 - Abstracts from hardware details
- **Westbound API:** between controllers 
 - Synchronization of network state information
 - E.g., coordinated flow setup, exchange of reachability information
- **Eastbound API:** interface to legacy infrastructures 
 - Usually proprietary

No widely accepted standard for north-bound / westbound interfaces currently exists. Controllers provide their own APIs.



IMPORTANT PRIMITIVES

Primitives for SDN Programming

- From **intended behavior** to **lower level flow rules**
- Three important areas to cover



(1) Create and install flow rules

(2) React to data plane events

(3) Inject individual packets

Create and Install Flow Rules

```
import ...

onConnect(switch) {
  if (switch == S1) {
    r = Rule()
    r.MATCH('IP_DST', 1.2.3.4)
    r.ACTION('OUTPUT', 4)
    send_rule(r, switch)
  }
}
```

static_forwarding.java

- onConnect
- Rule.MATCH
- Rule.ACTION
- send_rule

- entry point to implement custom logic
- select packets based on certain criteria
- specify what happens to matched packets
- install flow rule in switch

Overview: Matches

Usage	Description
<code>r.MATCH(IP_DST, 1.2.3.4)</code>	Match on a single IP destination address
<code>r.MATCH(IP_SRC, 1.2.3.4)</code> <code>r.MATCH(IP_DST, 9.8.7.6)</code>	Match can be used multiple times for more specific matching, e.g., on a pair of source and destination IP address
<code>r.MATCH(ETHERTYPE, ARP)</code>	Match all packets using a specific protocol inside the ethernet frame (here ARP)
<code>r.MATCH(MAC_DST, H1)</code>	Symbols may be used for parameters if it makes sense (e.g., H1 instead of 03:4b:3c:12:14:aa)
<code>r.MATCH(IP_DST, 1.0.0.0/8)</code>	Address fields usually support longest prefix matching
<code>r.MATCH(INPORT, 5)</code>	Matches all packets arriving on port 5
<code>r.MATCH(*)</code>	Match on everything, i.e., the most generic match possible

Overview: Actions

Usage	Description
<code>r.ACTION(DROP)</code>	Drop packet
<code>r.ACTION(FLOOD)</code>	Send packet to all ports („flooding“) except the one where it was received
<code>r.ACTION(CONTROLLER)</code>	Send packet to the controller
<code>r.ACTION(OUTPUT, 14)</code>	Send packet to port 14
<code>r.ACTION(GOTO, 4)</code>	If the switch has more than one flow table, the GOTO command is used to redirect a packet to another table (here flow table 4)
<code>r.ACTION(SET_FIELD, IP_SRC, 1.2.3.5)</code>	Overwrite IP source address of the packet with 1.2.3.5
<code>r.ACTION(SET_FIELD, TCP_DST, 40)</code>	Overwrite TCP destination port of the packet with 40

There are more/other matches and actions in a real SDN environment



Priorities

- Priorities come into play if there are **overlapping flow rules**
 - No overlap = all potential packets can only be matched by at most one rule
 - Overlap = at least one packet could be matched by more than one rule

- First rule

- `rule1.MATCH(IP_DST, 40.2.4.4)`
- `rule1.ACTION(DROP)`
- `rule1.PRIORITY(10)`

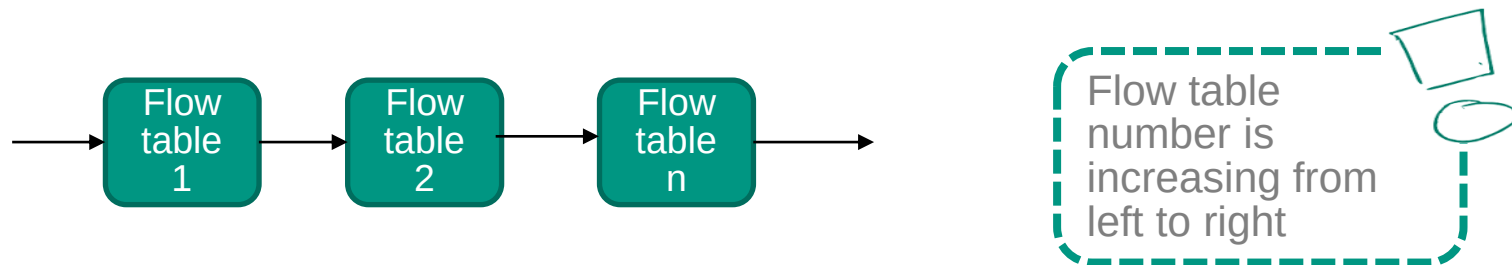
- Second rule

- `rule2.MATCH(IP_DST, 40.0.0.0/8)`
- `rule2.ACTION(OUTPUT, 1)`
- `// rule2.PRIORITY(1)`

`rule1.PRIORITY >
rule2.PRIORITY`

Multiple Flow Tables

- SDN switches can support **more than one flow table**
 - Common in software switches
 - Expensive in hardware (TCAM limited by available space / energy)

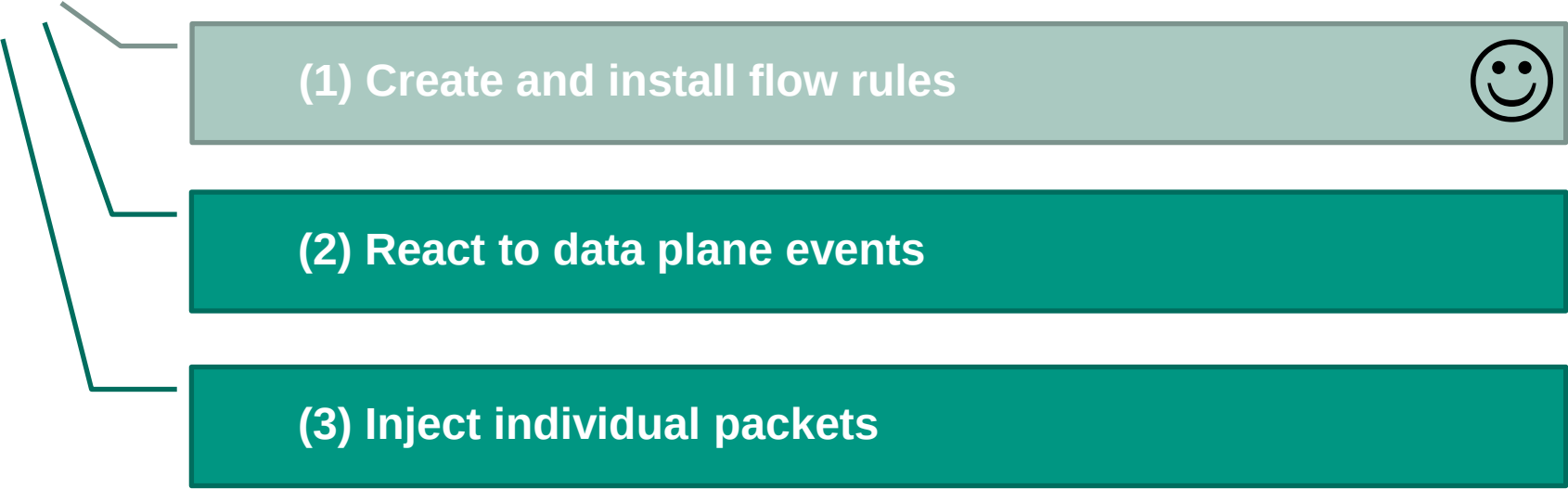


- Using multiple tables has several **benefits**
 - Can be used to isolate flow rules from different apps
 - Logical separation between different tasks (one table for monitoring, one table for security, ...)
 - In some situation: less overall flow table entries

Multiple Flow Tables

- `r.TABLE(x)` → specify the table for this rule
- `r.ACTION(GOTO, y)` → specify that processing continues in table y
- Can not go to lower flow table number to avoid cycles

Primitives for SDN Programming



(1) Create and install flow rules



(2) React to data plane events

(3) Inject individual packets

React to Data Plane Events

- Controller APIs provide callbacks to react to various control events
 - Switch state changes
 - Link state changes
 - Warnings and errors
 - ...

- **onConnect(switch)**

→ if control connection to switch is established

- **onPacketIn(packet, switch, inport)**

→ Called if the controller receives a packet that was forwarded via
r.**ACTION**(**CONTROLLER**)

Primitives for SDN Programming



(1) Create and install flow rules



(2) React to data plane events



(3) Inject individual packets

Inject Individual Packets

- `send_packet(packet, switch, rule)`
 - Injects a single packet into a switch

- This is different from installing flow rules
 - It is used for a single packet only
 - The flow table is not changed
 - Even if the `rule` parameter is present, this does NOT create a new flow table entry

Two Ways to Process Injected Packets

- Inject and process injected packet in the **flow table**

```
newPacket = createNewPacket()  
send_packet(newPacket, S1)
```

- Inject and process injected packet with a **custom rule**

```
newPacket = createNewPacket()  
customRule = Rule()  
customRule.ACTION(OUTPUT, 1)  
send_packet(newPacket, S1, customRule)
```


Primitives for SDN Programming



(1) Create and install flow rules



(2) React to data plane events



(3) Inject individual packets



Summary on Primitives

- Entry point primitives: Callbacks to implement custom logic
 - **onConnect(switch)**
 - Called if a new control connection to switch is established
 - **onPacketIn(packet, switch, port)**
 - Called if a packet was forwarded to the controller

- Flow rule creation primitives: Used to define flow rules
 - **Rule.MATCH()**
 - Select packets based on certain header fields
 - **Rule.ACTION()**
 - Specify what happens to a packet in the switch
 - **Rule.PRIORITY()**
 - Specify the priority of the created flow rule
 - **Rule.TABLE()**
 - Specify the flow table the rule should be applied to

Summary on Primitives

- Switch interaction primitives: Used to handle flow rule installation and packet injection
 - **send_rule(rule, switch)**
 - Installs a flow rule and creates the associated flow table entry in the switch
 - **send_packet(packet, switch)**
 - Injects a single packet into a switch, process with existing flow table entries
 - **send_packet(packet, switch, rule)**
 - Injects a single packet into a switch, process with custom rule

OPENFLOW

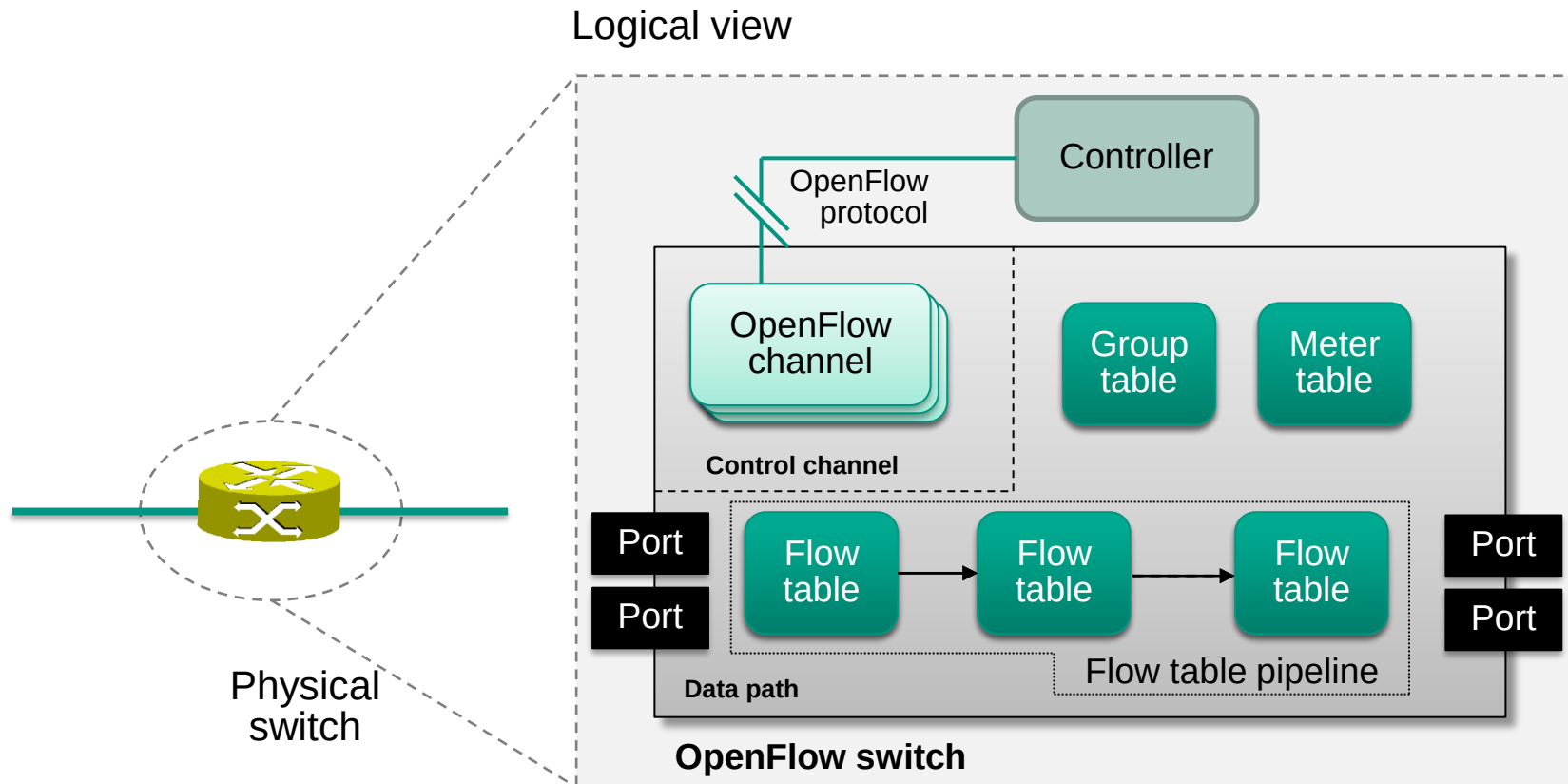
OpenFlow

- A standard for an SDN **southbound interface**
 - Defines the **interaction** between controller and switches
 - Defines a **logical architecture** for SDN switches (flow table, ...)
 - Defined by the Open Networking Foundation (ONF)
- Supports all basic structures and primitives seen so far
 - Matches
 - Actions
 - Priorities
 - Multiple flow tables
 - Protocol mechanisms for
 - Creating flow rules
 - Reacting to data plane events
 - Injecting individual packets
- And more sophisticated features
 - Group table, rate limiting



OpenFlow Switch Architecture

- Provides a uniform view on SDN-capable switches



Flow Table in OpenFlow

- Next to match (+ priority) and action, there are several other important fields in an OpenFlow flow table



- Counters
 - The number of processed packets (**counter**)
- Timeouts
 - Maximum lifetime of a flow
 - Enables automatic removal of flows
- Cookie
 - Marker value set by an SDN controller
 - Not used during packet processing
 - Simplifies flow management
- Flags
 - Indicate how a flow is managed
 - E.g., notify controller when a flow is automatically removed

Group Tables

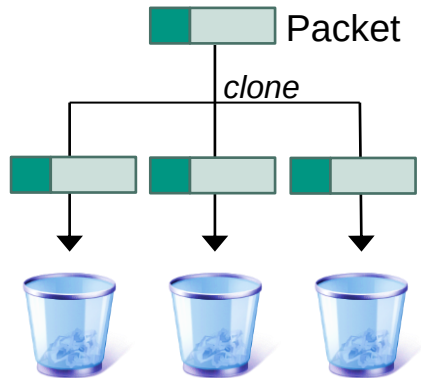
Group entry

Group identifier	Group type	Counters	Action buckets
------------------	------------	----------	----------------

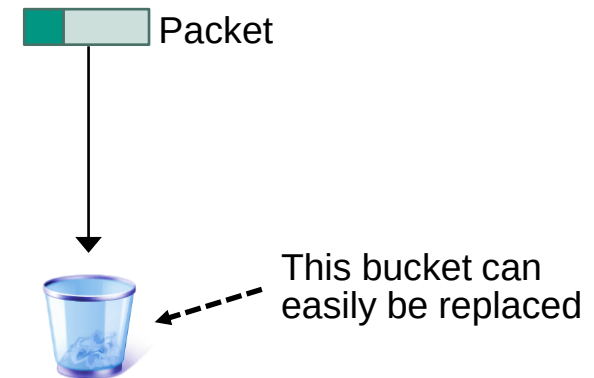
- **Group tables** represent additional forwarding methods
 - E.g., link selection, fast failover, ...
- **Group entries** can be invoked from other tables via **group actions**
 - They are referenced by their unique **group identifier**
 - Flow table entries can perform group actions during ingress processing
- Effect of group processing depends on the **group type** and its **action buckets**

Group Types

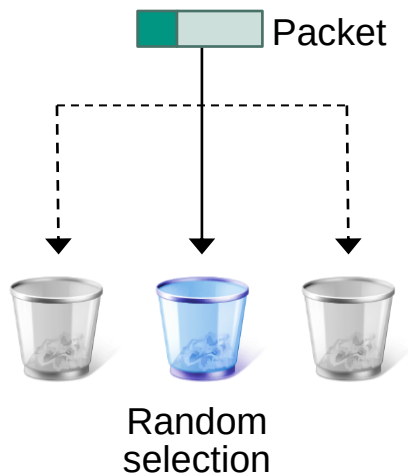
Type = **all**



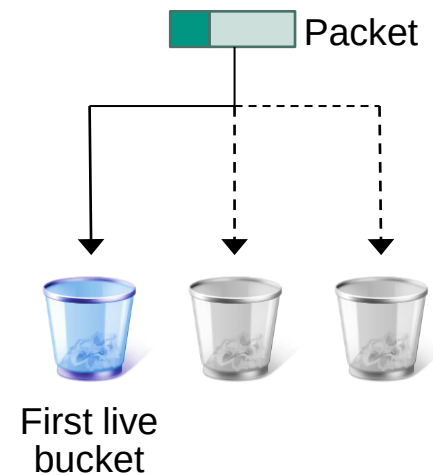
Type = **indirect**



Type = **select**



Type = **fast failover**



Group Types

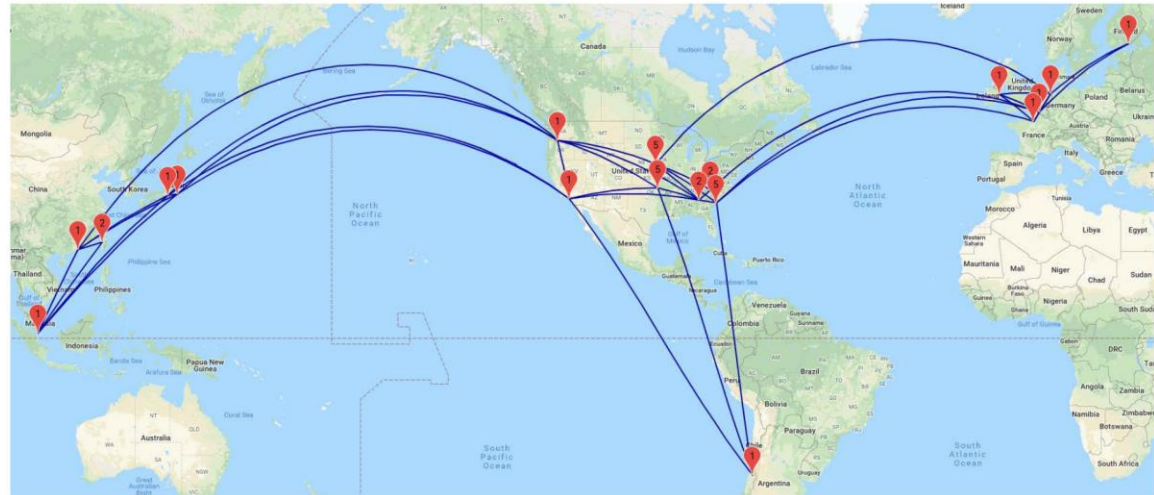
■ Group types have specific semantics

- **All:** executes **all buckets** in a group
 - E.g., for broadcast
- **Select:** selects **one of many buckets** of a group
 - E.g., select by round-robin or hashing of packet data
- **Fast failover:** executes **first live bucket** in a group
 - Each bucket is associated with a port that determines its liveliness
 - E.g., a bucket is live when its associated port is up
- **Indirect:** executes **the single bucket** in a group
 - Indirect groups must reference exactly **one** action bucket
 - Useful to avoid changing multiple flow table entries with common actions

SDN IN THE FIELD

SDN in private inter-datacenter WANs

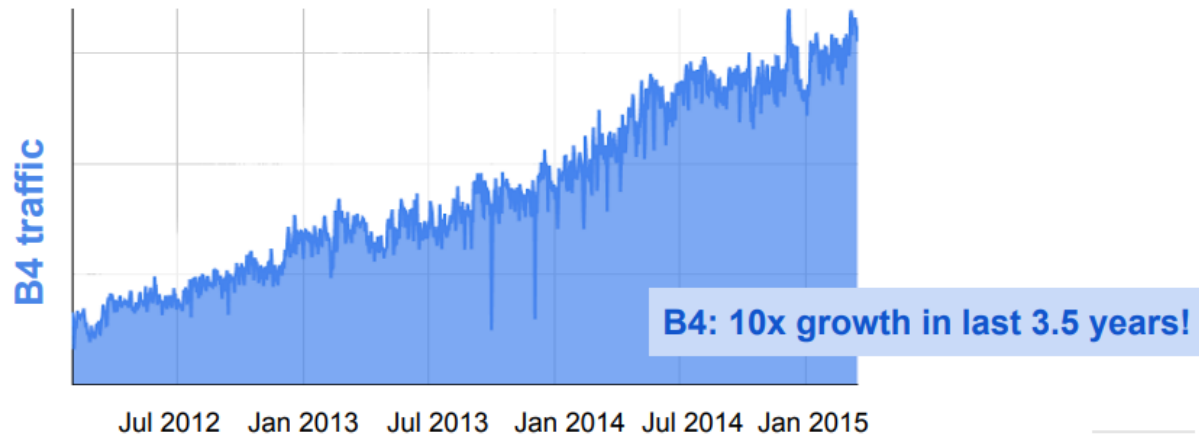
- Google deployed SDN for its inter-datacenter network (B4) in 2011
- **Large scale** example for traffic engineering with SDN
 - ~20% increase in throughput over shortest path forwarding
 - Easy software upgrades and new features
 - Slower HW upgrades → 3 generations of HW under same SDN architecture



- Similar solutions by Microsoft (SWAN), Meta (EBB), and more ...

Google B4 & More

- B4 still in place
 - Handled traffic volume increases (data from 2015)



- Other examples from Google: **Jupiter**
 - SDN-based data center network
 - 1 Petabit/s of total bisection bandwidth
 - That means: 100,000 servers can exchange information at 10Gb/s each
- Similar examples available from Facebook, Microsoft, ...

LITERATURE



- [BCA+13] M. F. Bari, S. R. Chowdhury, R. Ahmed and R. Boutaba; [PolicyCop: An Autonomic QoS Policy Enforcement Framework for Software Defined Networks](#); 2013 IEEE SDN for Future Networks and Services (SDN4FNS), Trento, 2013, pp. 1-7.
- [BRKB13] F. A. Botelho, F. M. V. Ramos, D. Kreutz and A. N. Bessani; [On the Feasibility of a Consistent and Fault-Tolerant Data Store for SDNs](#); 2013 Second European Workshop on Software Defined Networks, 2013, pp. 38-43.
- [CaSS14] C.J. Casey, A. Sutton, A. Sprintson; [tinyNBI: Distilling an API from Essential OpenFlow Abstractions](#); Proceedings of the Third Workshop on Hot Topics in Software Defined Networking 2014
- [FHF+11] N. Foster, R. Harrison, M. J. Freedman, C. Monsanto, J. Rexford, A. Story, and D. Walker; [Frenetic: a network programming language](#); SIGPLAN Not. 46, 9 (September 2011), 279-291.
- [GHB+17] Gebert, S., Herrnleben, S., Bauer, R., Geissler, S., Zinner, T. and, Jarschel, M.: [TableVisor 2.0: Towards Full-Featured, Scalable and Hardware-Independent Multi Table Processing](#). NetSoft 2017
- [GiLy02] Seth Gilbert and Nancy Lynch. 2002. [Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services](#). SIGACT News 33, 2 (June 2002), 51-59.
- [HSF+09a] Handigol, Nikhil & Seetharaman, Srini & Flajslik, Mario & McKeown, Nick & Johari, Ramesh. (2009). [Plug-n-Serve: Load-Balancing Web Traffic using OpenFlow](#). ACM Sigcomm Demo.

- [HSF+09b] Handigol, N., Seetharaman, S., Flajslik, M., Gember, A., McKeown, N., Parulkar, G., Akella, A., Feamster, N., Clark, R., Krishnamurthy, A., Brajkovic, V. & Anderson, T. (2009). [Aster*x: Load-Balancing Web Traffic over Wide-Area Networks](#) (Technical report, Stanford University)
- [GiLy02] Seth Gilbert and Nancy Lynch. 2002. [Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services](#). SIGACT News 33, 2 (June 2002), 51-59.
- [Jain13] S. Jain et al.; [B4: Experience with a globally-deployed software defined WAN](#); Proceedings of the ACM SIGCOMM 2013
- [JZH+14] M. Jarschel, T. Zinner, T. Hossfeld, P. Tran-Gia and W. Kellerer; [Interfaces, attributes, and use cases: A compass for SDN](#), in IEEE Communications Magazine, vol. 52, no. 6, pp. 210-217, June 2014
- [KaDu17] Murat Karakus, Arjan Duresi, [A survey: Control plane scalability issues and approaches in Software-Defined Networking \(SDN\)](#), In Computer Networks, Volume 112, 2017, Pages 279-293
- [KRVR+15] D. Kreutz, F.M.V. Ramos, P. Esteves Verissimo, C. Esteve Rothenberg, S. Azodolmolky, S. Uhlig; [Software-Defined Networking: A Comprehensive Survey](#); Proceedings of the IEEE, Vol103, Issue 1, Jan 2015, pp.14-76
- [LHM10] Bob Lantz, Brandon Heller, and Nick McKeown. 2010. [A network in a laptop: rapid prototyping for software-defined networks](#). In Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks, ACM, New York
- [ODL17] [OpenDaylight Controller: Architectural Framework](#); 2017;
https://wiki.opendaylight.org/view/OpenDaylight_Controller:Architectural_Frame

- [ONF14] Open Networking Foundation; [OpenFlow Switch Specification, Version 1.5.1](https://www.opennetworking.org/software-defined-standards/specifications/); Apr 2014; <https://www.opennetworking.org/software-defined-standards/specifications/>
- [PPK+15] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan J. Jackson, Andy Zhou, Jarno Rajahalme, Jesse Gross, Alex Wang, Jonathan Stringer, Pravin Shelar, Keith Amidon, and Martín Casado. 2015. [The design and implementation of open vSwitch](#). In Proceedings of the 12th USENIX Conference on Networked Systems Design and Implementation (NSDI'15). USENIX Association, Berkeley, CA, USA, 117-130.
- [PSG+13] Aurojit Panda, Colin Scott, Ali Ghodsi, Teemu Koponen, and Scott Shenker. 2013. [CAP for networks](#). In Proceedings of the second ACM SIGCOMM workshop on Hot topics in software defined networking (HotSDN '13). ACM, New York, NY, USA, 91-96.
- [RFR+12] Mark Reitblatt, Nate Foster, Jennifer Rexford, Cole Schlesinger, and David Walker. 2012. [Abstractions for network update](#). In Proceedings of the ACM SIGCOMM 2012. ACM, New York
- [RFC7426] [Software-Defined Networking \(SDN\): Layers and Architecture Terminology](#); IETF, RFC 7426, Jan 2015
- [SCK+11] S. Shenker, M. Casado, T. Koponen, N. McKeown; [The Future of Networking, and the Past of Protocols](#); 2011; <https://www.cs.princeton.edu/events/event/future-networking-and-past-protocols>
- [SCS13] S. Scott-Hayward, G. O'Callaghan and S. Sezer, "[Sdn Security: A Survey](#)," 2013 IEEE SDN for Future Networks and Services (SDN4FNS), Trento, 2013, pp. 1-7.

- [SGY+09] Rob Sherwood and Glen Gibb and Kok-kiong Yap and Martin Casado and Nick Mckeown and Guru Parulkar, [FlowVisor: A Network Virtualization Layer](#), Technical Report, 2009
- [Stal15] W.Stallings; [Foundations of Modern Networking: SDN, NFV, QoE, IoT, and Cloud](#); Addison-Wesley Professional; 1 edition; 2015
- [ToGa10] A. Tootoonchian, Y. Ganjali. 2010; [HyperFlow: a distributed control plane for OpenFlow](#); Proceedings of the 2010 internet network management conference on Research on enterprise networking (INM/WREN'10). USENIX Association, 2010, pp. 3-3
- [WBR11] Richard Wang, Dana Butnariu, and Jennifer Rexford. 2011. [OpenFlow-based server load balancing gone wild](#). In Proceedings of the 11th USENIX conference on Hot topics in management of internet, cloud, and enterprise networks and services (Hot-ICE'11). USENIX Association, Berkeley, CA, USA, 12-12.